

DOLLOR.AI

Complete System Architecture & API Documentation

Customer App | Driver App | Restaurant App | Rideshare

December 2024 | Version 2.0

API Base URL: <https://api.dollor.ai/api>

Status: **PRODUCTION READY**

1. SYSTEM OVERVIEW

COMPONENT	TECHNOLOGY	DESCRIPTION	KEY FEATURES
Customer App (com.dollor.customer)	SwiftUI iOS 17+	Food ordering & Rideshare booking	• Browse restaurants • Place orders • Track delivery • Book rides • Apple/Google Pay
Driver App (com.dollor.driver)	SwiftUI iOS 17+	Delivery & Ride fulfillment	• Accept deliveries • Accept rides • GPS navigation • Earnings tracking • Document upload
Restaurant App (com.dollor.restaurant)	SwiftUI iOS 17+	Order & Menu management	• Receive orders • Manage menu • Set availability • View analytics • Create promotions
Backend API	FastAPI Python 3.11	136 REST endpoints	• JWT authentication • Rate limiting • Auto-scaling • Real-time events
Database	PostgreSQL AWS RDS	Data persistence	• 15+ tables • Relationships • Indexes • Backups
Payments	Stripe Connect	P2P payment processing	• Payment intents • Direct transfers • \$1-3 tiered fees • Instant payouts
Authentication	JWT + Firebase	User authentication	• Email/password • Google Sign-In • Apple Sign-In • 24hr token expiry

Maps	Google Maps SDK	Location services	• Restaurant maps • Driver tracking • Route navigation • Distance calc
Notifications	Firebase FCM	Push notifications	• Order updates • Delivery alerts • New orders • Promotions

2. COMPLETE SYSTEM ARCHITECTURE

How all components connect together:



■	CUSTOMER APP	■
■	(iOS 17+)	■
■		■
■	• HomeView	■
■	• RestaurantDetail	■
■	• CartView	■
■	• CheckoutView	■
■	• OrderTracking	■
■	• TripBoardView	■
■	• ProfileView	■
■		
■	DRIVER APP	■
■	(iOS 17+)	■
■		■
■	• DashboardView	■
■	• AvailableOrders	■
■	• ActiveDelivery	■
■	• AvailableRides	■
■	• EarningsView	■
■	• DocumentsView	■
■		
■	RESTAURANT APP	■
■	(iOS 17+)	■
■		■
■	• OrdersView	■
■	• MenuManagement	■
■	• PromotionsView	■
■	• AnalyticsView	■
■	• SettingsView	■

[illegible][illegible]

3. CUSTOMER APP - ALL API ENDPOINTS

CATEGORY	METHOD	ENDPOINT	REQUEST BODY	RESPONSE	DESCRIPTION
Authentication	POST	/api/customer/login	{ "email": "string", "password": "string" }	{ "access_token": "jwt...", "customer_id": 1, "name": "John", "email": "john@email.com" }	Login with email/password
Authentication	POST	/api/customer/register	{ "email": "string", "password": "string", "full_name": "string", "phone": "string" }	{ "access_token": "jwt...", "customer_id": 1 }	Register new customer
Authentication	POST	/api/customer/google-auth	{ "email": "string", "name": "string", "google_id": "string" }	{ "access_token": "jwt...", "customer_id": 1 }	Google Sign-In
Authentication	POST	/api/customer/apple-auth	{ "email": "string", "name": "string", "apple_id": "string" }	{ "access_token": "jwt...", "customer_id": 1 }	Apple Sign-In
Restaurants	GET	/api/public/restaurants	None	[{ "id": 1, "name": "Pizza Place", "cuisine": "Italian", "rating": 4.5 }]	Get all restaurants

Restaurants	GET	/api/public/restaurants/{id}	None	{ "id": 1, "name": "...", "menu": [...], "hours": {...} }	Get restaurant details
Menu	GET	/api/vendors/{id}/menu	None	[{ "id": 1, "name": "Pepperoni", "price": 15.99, "category": "Pizza" }]	Get restaurant menu
Orders	POST	/api/orders/create	{ "customer_id": 1, "vendor_id": 1, "items": [...], "address": "...", "payment_intent": "pi..." }	{ "order_id": 123, "status": "pending", "total": 45.99 }	Create new order
Orders	GET	/api/customer/orders	Header: Authorization: Bearer {token}	[{ "id": 1, "status": "delivered", "total": 45.99 }]	Get customer orders
Tracking	GET	/api/customer/orders/{id}/track	None	{ "status": "picked_up", "driver": {...}, "eta": "15 min" }	Track order status
Tracking	GET	/api/erp/orders/{id}/driver-location	None	{ "latitude": 34.0522, "longitude": -118.2437, "heading": 90 }	Get driver GPS location

Payments	POST	/api/payments/create-intent	{ "amount": 4599, "currency": "usd" }	{ "clientSecret": "pi_...", "publishableKey": "pk_..." }	Create Stripe payment
----------	------	-----------------------------	--	---	-----------------------

4. CUSTOMER APP - RIDESHARE API ENDPOINTS

CATEGORY	METHOD	ENDPOINT	REQUEST BODY	RESPONSE	DESCRIPTION
Request Ride	POST	/api/rides/request	<pre>{ "customer_id": 1, "pickup_lat": 34.0522, "pickup_lng": -118.2437, "pickup_address": "123 Main St", "dropoff_lat": 34.0622, "dropoff_lng": -118.2537, "dropoff_address": "456 Oak Ave" }</pre>	<pre>{ "ride_id": 1, "status": "pending", "estimated_fare": 15.50, "estimated_time": "8 min" }</pre>	Request a new ride
My Rides	GET	/api/customer/rides	Header: Authorization: Bearer {token}	<pre>{ "rides": [{ "id": 1, "status": "completed", "fare": 15.50, "driver_name": "John" }], "count": 1 }</pre>	Get all customer rides
Track Ride	GET	/api/rides/{id}/track	None	<pre>{ "ride_id": 1, "status": "in_progress", "driver": { "name": "John", "vehicle": "Toyota Camry", "plate": "ABC123" }, "driver_location": { "lat": 34.0525, "lng": -118.2440 }, "eta": "3 min" }</pre>	Track ride in real-time

Cancel Ride	POST	/api/rides/{id}/cancel	{ "reason": "Changed plans" }	{ "success": true, "message": "Ride cancelled" }	Cancel a pending ride
Rate Driver	POST	/api/rides/{id}/rate	{ "rating": 5, "comment": "Great driver!" }	{ "success": true }	Rate driver after ride
Fare Estimate	POST	/api/rides/estimate	{ "pickup_lat": 34.0522, "pickup_lng": -118.2437, "dropoff_lat": 34.0622, "dropoff_lng": -118.2537 }	{ "distance_miles": 2.5, "estimated_fare": 15.50, "estimated_time": "8 min", "breakdown": { "base": 2.50, "distance": 3.75, "platform_fee": 1.00 } }	Get fare estimate before booking

5. DRIVER APP - ALL API ENDPOINTS

CATEGORY	METHOD	ENDPOINT	REQUEST BODY	RESPONSE	DESCRIPTION
Authentication	POST	/api/auth/driver/login	Form Data: username=email password=pwd	{ "access_token": "jwt...", "driver_id": 1, "name": "John Driver" }	Login (form-urlencoded)
Authentication	POST	/api/auth/driver/register	{ "email": "string", "password": "string", "first_name": "string", "last_name": "string", "phone": "string" }	{ "access_token": "jwt...", "driver_id": 1 }	Register new driver
Authentication	POST	/api/auth/driver/google	{ "id_token": "string", "email": "string", "name": "string" }	{ "access_token": "jwt...", "driver_id": 1 }	Google Sign-In
Profile	GET	/api/erp/drivers/{id}	None	{ "id": 1, "name": "John", "vehicle": "Toyota", "rating": 4.8, "is_online": true }	Get driver profile
Profile	PUT	/api/erp/drivers/{id}	{ "vehicle_type": "Sedan", "vehicle_color": "Black", "license_plate": "ABC123" }	{ "success": true }	Update driver profile

Documents	GET	/api/drivers/{id}/documents	None	{ "documents": [{ "type": "license", "status": "approved" }] }	Get uploaded documents
Documents	POST	/api/drivers/{id}/documents	Multipart Form: file=<binary> type=license	{ "document_id": 1, "status": "pending" }	Upload document
Deliveries	GET	/api/v2/driver/deliveries/available	None	[{ "order_id": 1, "restaurant": "Pizza Place", "earnings": 8.50, "distance": 2.3 }]	Get available deliveries
Deliveries	POST	/api/v2/driver/deliveries/{id}/accept	{}	{ "success": true, "order": {...} }	Accept a delivery
Deliveries	POST	/api/v2/driver/deliveries/{id}/pickup	{}	{ "success": true, "status": "picked_up" }	Mark order picked up
Deliveries	POST	/api/v2/driver/deliveries/{id}/complete	{}	{ "success": true, "earnings": 8.50 }	Complete delivery
Location	POST	/api/driver/location	{ "driver_id": 1, "latitude": 34.0522, "longitude": -118.2437, "heading": 90 }	{ "success": true }	Update GPS location

Earnings	GET	/api/drivers/{id}/earnings	None	{ "today": 85.50, "week": 450.00, "month": 1800.00, "pending": 25.00 }	Get earnings summary
Dashboard	GET	/api/v2/driver/dashboard/{id}	None	{ "stats": {...}, "earnings": {...}, "active_delivery": {...} }	Get dashboard data

6. DRIVER APP - RIDESHARE API ENDPOINTS

CATEGORY	METHOD	ENDPOINT	REQUEST BODY	RESPONSE	DESCRIPTION
Available Rides	GET	/api/v2/driver/rides/available	None	[{ "ride_id": 1, "pickup_address": "123 Main St", "dropoff_address": "456 Oak Ave", "distance_miles": 5.2, "estimated_fare": 18.50, "customer_name": "John" }]	Get nearby ride requests
Accept Ride	POST	/api/v2/driver/rides/{id}/accept	{}	{ "success": true, "ride": { "id": 1, "customer_name": "John", "customer_phone": "555-1234", "pickup_address": "123 Main St", "pickup_lat": 34.0522, "pickup_lng": -118.2437 } }	Accept a ride request
Arrive at Pickup	POST	/api/v2/driver/rides/{id}/arrive	{}	{ "success": true, "status": "driver_arrived", "message": "Customer notified" }	Mark arrival at pickup
Start Ride	POST	/api/v2/driver/rides/{id}/start	{}	{ "success": true, "status": "in_progress", "started_at": "2024-12-12T10:30:00Z" }	Start the ride (customer in car)

Complete Ride	POST	/api/v2/driver/rides/{id}/complete	{}	{ "success": true, "status": "completed", "earnings": 15.50, "tip": 3.00, "total_earned": 18.50 }	Complete the ride
Cancel Ride	POST	/api/v2/driver/rides/{id}/cancel	{ "reason": "Customer no show" }	{ "success": true, "cancellation_fee": 5.00 }	Cancel an accepted ride
Ride History	GET	/api/erp/driver/{id}/rides	None	{ "rides": [{ "id": 1, "date": "2024-12-12", "earnings": 18.50, "distance": 5.2, "rating": 5 }], "total_rides": 150, "total_earnings": 2500.00 }	Get ride history
Go Online	POST	/api/drivers/{id}/status	{ "is_online": true, "driver_type": "both" }	{ "success": true, "is_online": true }	Set online status (both = delivery + rideshare)

7. RESTAURANT APP - ALL API ENDPOINTS

CATEGORY	METHOD	ENDPOINT	REQUEST BODY	RESPONSE	DESCRIPTION
Authentication	POST	/api/auth/vendor/login	Form Data: username=email password=pwd	{ "access_token": "jwt...", "vendor_id": 1, "business_name": "Pizza Place" }	Login (form-urlencoded)
Authentication	POST	/api/auth/vendor/register	{ "email": "string", "password": "string", "business_name": "string", "phone": "string", "address": "string", "city": "string", "state": "string", "zip_code": "string" }	{ "access_token": "jwt...", "vendor_id": 1 }	Register new restaurant
Orders	GET	/api/erp/orders/vendor/{id}	None	[{ "order_id": 1, "customer": "John", "items": [...], "total": 45.99, "status": "pending" }]	Get all orders
Orders	POST	/api/erp/orders/{id}/accept	{}	{ "success": true, "status": "confirmed", "prep_time": 20 }	Accept an order
Orders	POST	/api/erp/orders/{id}/reject	{ "reason": "Item unavailable" }	{ "success": true, "refund_initiated": true }	Reject an order

Orders	POST	/api/erp/orders/{id}/ready	{}	{ "success": true, "status": "ready", "driver_notified": true }	Mark order ready for pickup
Menu	GET	/api/vendors/{id}/menu	None	[{ "id": 1, "name": "Pepperoni Pizza", "price": 15.99, "available": true }]	Get menu items
Menu	POST	/api/vendors/{id}/menu	{ "name": "New Item", "description": "...", "price": 12.99, "category": "Appetizers" }	{ "item_id": 5, "success": true }	Add menu item
Menu	PUT	/api/vendors/{id}/menu/{item_id}	{ "price": 14.99, "is_available": false }	{ "success": true }	Update menu item
Menu	DELETE	/api/vendors/{id}/menu/{item_id}	None	{ "success": true }	Delete menu item
Promotions	GET	/api/promotions/vendor/{id}	None	[{ "id": 1, "name": "20% Off", "type": "percentage", "value": 20 }]	Get active promotions

Promotions	POST	/api/promotions/vendor/{id}	{ "name": "Happy Hour", "discount_type": "percentage", "discount_value": 15, "start_date": "...", "end_date": "..." }	{ "promo_id": 2, "success": true }	Create promotion
Analytics	GET	/api/promotions/analytics/{id}	None	{ "total_orders": 150, "revenue": 5000.00, "avg_order": 33.33, "top_items": [...] }	Get analytics

8. RIDESHARE - COMPLETE END-TO-END FLOW

STEP	CUSTOMER APP	BACKEND API	DRIVER APP	DATABASE CHANGES
1. Request Ride	TripBoardView: • Enter pickup location • Enter destination • See fare estimate • Tap "Request Ride"	POST /api/rides/request Calculates: • Distance (Haversine) • Fare = \$2.50 base + \$1.50/mile + \$1-3 platform fee	(waiting)	INSERT INTO rides: • status = "pending" • customer_id • pickup_lat/lng • dropoff_lat/lng • estimated_fare
2. Find Driver	Shows: "Finding driver..."	Query drivers WHERE: • is_online = true • driver_type IN ("rideshare", "both") • distance < 5 miles Send FCM push to nearby drivers	Push notification: "New ride: \$18.50 5.2 miles away"	No changes yet
3. Driver Accepts	Shows: "Driver found!" • Driver name • Vehicle info • ETA to pickup	POST /api/v2/driver/rides/{id}/accept Assign driver to ride Send FCM to customer	Tap "Accept" Shows: • Customer name • Pickup address • Navigation button	UPDATE rides SET: • driver_id = X • status = "accepted"
4. Driver En Route	Live map shows: • Driver location • ETA updating Polls every 5 sec: GET /api/rides/{id}/track	POST /api/driver/location (every 10 seconds) Stores driver GPS	Navigating to pickup Auto-sends location every 10 seconds	UPDATE drivers SET: • current_lat = X • current_lng = Y
5. Driver Arrives	Push notification: "Driver arrived!" Shows: • Driver waiting • Vehicle details	POST /api/v2/driver/rides/{id}/arrive Send FCM to customer	Tap "Arrived" Waiting for customer	UPDATE rides SET: • status = "arrived"

6. Start Ride	Shows: • "In transit" • Live route • ETA to destination	POST /api/v2/driver/rides/{id}/start	Tap "Start Ride" (customer in car) Navigation to dropoff	UPDATE rides SET: • status = "in_progress" • started_at = NOW()
7. Complete Ride	Shows: • "Ride completed!" • Final fare • Tip option • Rate driver (1-5 stars)	POST /api/v2/driver/rides/{id}/complete Process payment: • Charge customer • Transfer to driver • Platform keeps fee	Tap "Complete" Shows: • Earnings added • Back to available	UPDATE rides SET: • status = "completed" • completed_at = NOW() • final_fare = X INSERT payment record
8. Payment Distribution	Stripe charges card	Stripe Connect: Fare: \$18.50 ■■ Driver: \$15.50 (84%) ■■ Platform: \$2.00 ■■ Stripe: \$1.00 (fees)	Earnings updated: +\$15.50 + tip	UPDATE drivers SET: • total_earnings += X INSERT INTO payouts

9. FOOD DELIVERY - COMPLETE END-TO-END FLOW

STEP	CUSTOMER APP	BACKEND API	RESTAURANT APP	DRIVER APP	DATABASE
1. Browse	HomeView: - Browse restaurants - View menu - Add to cart	GET /api/public/ restaurants GET /api/vendors/ {id}/menu	(waiting for orders)	(offline or available)	SELECT FROM vendors, menu_items
2. Checkout	CheckoutView: - Review cart - Enter address - Add tip - Apple/Google Pay	POST /api/payments/ create-intent Stripe PaymentIntent	(waiting)	(waiting)	No changes yet
3. Place Order	Tap "Place Order" Payment confirmed	POST /api/orders/create Create order record Send FCM to restaurant	PUSH: "New Order!" OrdersView shows new order	(waiting)	INSERT INTO orders: - status = "pending" - items, total - customer_id - vendor_id
4. Restaurant Accepts	Shows: "Order confirmed" "Preparing..."	POST /api/erp/orders/ {id}/accept Send FCM to customer	Tap "Accept" Start preparing food	(waiting)	UPDATE orders SET: status = "confirmed"
5. Order Ready	Shows: "Finding driver..."	POST /api/erp/orders/ {id}/ready Find nearby drivers Send FCM to drivers	Tap "Ready" Waiting for pickup	PUSH: "New delivery!" \$8.50 • 2.3 mi	UPDATE orders SET: status = "ready"
6. Driver Accepts	Shows: "Driver assigned" - Driver name - Vehicle - ETA	POST /api/v2/driver/ deliveries/{id}/accept Assign driver Send FCM to all	Shows: "Driver en route" Driver name	Tap "Accept" Navigate to restaurant	UPDATE orders SET: - driver_id = X - status = "assigned"
7. Pickup	Shows: "Driver picking up your order"	POST /api/v2/driver/ deliveries/{id}/pickup	Driver arrives Hands over food	Tap "Picked Up" Navigate to customer	UPDATE orders SET: status = "picked_up"

8. Delivery	Live tracking: • Driver location • ETA countdown "Driver arriving!"	POST /api/driver/location (every 10 sec) GET /api/erp/orders/{id}/driver-location	Order complete for restaurant	Navigate to customer address	UPDATE drivers SET: • current_lat • current_lng
9. Complete	PUSH: "Delivered!" Rate & tip option	POST /api/v2/driver/deliveries/{id}/complete Process payments	Analytics updated	Tap "Delivered" Earnings: +\$8.50	UPDATE orders SET: • status = "delivered" • delivered_at

10. PAYMENT FLOW & TIERED PRICING

ORDER TOTAL	CUSTOMER PAYS	RESTAURANT PAYS	DRIVER GETS	PLATFORM GETS
≤ \$35 (Tier 1)	Food + \$1 fee + Tax + Tip	\$1 platform fee (from food total)	Base \$3 + \$0.50/mile + 100% tip	\$2 total (\$1 + \$1)
\$35.01 - \$70 (Tier 2)	Food + \$2 fee + Tax + Tip	\$2 platform fee (from food total)	Base \$4 + \$0.60/mile + 100% tip	\$4 total (\$2 + \$2)
> \$70 (Tier 3)	Food + \$3 fee + Tax + Tip	\$3 platform fee (from food total)	Base \$5 + \$0.75/mile + 100% tip	\$6 total (\$3 + \$3)

EXAMPLE: \$50 Order Calculation

ITEM	AMOUNT	WHO PAYS	WHO RECEIVES
Food Subtotal	\$50.00	Customer	Restaurant
Platform Fee (Tier 2)	\$2.00	Customer	Platform
Restaurant Fee (Tier 2)	\$2.00	Restaurant	Platform
Tax (8.25%)	\$4.13	Customer	Collected by Platform
Driver Tip	\$5.00	Customer	Driver (100%)
Driver Base Pay	\$4.00	Platform	Driver
Driver Mileage (3 mi)	\$1.80	Platform	Driver
CUSTOMER TOTAL	\$61.13		
RESTAURANT RECEIVES	\$48.00		(\$50 - \$2 fee)
DRIVER RECEIVES	\$10.80		(\$4 + \$1.80 + \$5 tip)
PLATFORM RECEIVES	\$4.00		(\$2 + \$2 fees)

11. DATABASE SCHEMA - ALL TABLES

TABLE	KEY COLUMNS	RELATIONSHIPS	PURPOSE
customers	id (PK) email (unique) password_hash full_name phone google_id apple_id stripe_customer_id created_at	→ orders (1:N) → rides (1:N) → addresses (1:N) → favorites (1:N)	Store customer account data
drivers	id (PK) email (unique) password_hash first_name, last_name phone vehicle_type, color license_plate is_online driver_type current_lat, current_lng stripe_account_id	→ orders (1:N) → rides (1:N) → documents (1:N)	Store driver account data and real-time location
vendors	id (PK) email (unique) password_hash business_name phone address, city, state, zip latitude, longitude cuisine_type is_active rating stripe_account_id	→ orders (1:N) → menu_items (1:N) → promotions (1:N)	Store restaurant account data

orders	id (PK) customer_id (FK) vendor_id (FK) driver_id (FK) status order_total platform_fee delivery_fee tax, tip grand_total delivery_address created_at	← customers ← vendors ← drivers → order_items (1:N)	Track all food delivery orders
rides	id (PK) customer_id (FK) driver_id (FK) status pickup_lat, pickup_lng pickup_address dropoff_lat, dropoff_lng dropoff_address distance_miles fare_amount platform_fee driver_earnings created_at started_at completed_at	← customers ← drivers	Track all rideshare trips
menu_items	id (PK) vendor_id (FK) name description price category image_url is_available is_vegetarian is_vegan prep_time_minutes	← vendors	Restaurant menu items

promotions	id (PK) vendor_id (FK) name discount_type discount_value minimum_order promo_code start_date, end_date is_active usage_count	← vendors	Restaurant promotions and deals
------------	---	-----------	---------------------------------------

12. DEMO CREDENTIALS & API REFERENCE

APP	EMAIL	PASSWORD	BUNDLE ID
Customer App	demo@dollar.ai	DollarDemo2024!	com.dollar.customer
Driver App	demodriver@dollar.ai	DollarDriver2024!	com.dollar.driver
Restaurant App	demobusiness@dollar.ai	DollarBiz2024!	com.dollar.restaurant

API ENDPOINTS SUMMARY

CATEGORY	COUNT	BASE PATH	AUTH REQUIRED
Customer Auth	5	/api/customer/*	No (login/register)
Customer Orders	8	/api/customer/*, /api/orders/*	Yes (JWT)
Customer Rides	6	/api/rides/*, /api/customer/rides	Yes (JWT)
Driver Auth	5	/api/auth/driver/*	No (login/register)
Driver Deliveries	10	/api/v2/driver/deliveries/*	Yes (JWT)
Driver Rides	8	/api/v2/driver/rides/*	Yes (JWT)
Driver Profile	6	/api/drivers/*, /api/erp/drivers/*	Yes (JWT)
Restaurant Auth	4	/api/auth/vendor/*	No (login/register)
Restaurant Orders	6	/api/erp/orders/*	Yes (JWT)
Restaurant Menu	5	/api/vendors/{id}/menu/*	Yes (JWT)
Restaurant Promos	4	/api/promotions/*	Yes (JWT)
Public	8	/api/public/*	No
Payments	3	/api/payments/*	Yes (JWT)
TOTAL	136		

API Base URL: <https://api.dollar.ai/api>

API Documentation: <https://api.dollar.ai/docs>

Legal Pages: <https://api.dollor.ai/terms> | <https://api.dollor.ai/privacy>

Platform Status: PRODUCTION READY